FLOPS

Подсчет для каждого слоя.

1) Conv2d ($H\_in, H\_out, k, st, p, B, S$)
   - bias = False во всех слоях, поэтому без него
   - Стоимость одной свертки с ядром $k\times k$ $= 2k^2$

$$\boxed{B\times \underbrace{H\_in\times H\_out}_{\text{для каждой пары каналов}}\times \underbrace{\left\lfloor \frac{S+2p-k}{st}+1\right\rfloor^2}_{\text{размер выхода для } S\times S \text{ входа}}\times \underbrace{2k^2}_{\text{одна свертка}}}$$

2) Linear (in_f, out_f, b=True, B)
   - $y = x^TW + b$; matvec + vec ⊜

⊜ $\boxed{B\times[(2\times in\_f-1)\times out\_f + out\_f]}$

3) BatchNorm2d
   - $y = \gamma\left(\frac{x-\mu}{\sqrt{\sigma^2+\varepsilon}}\right)+\beta$ (два аффинных преобр.)
   - input shape ($B\times H_{in}\times S\times S$), делаем для каждого элемента 4 FLOPs ⇒
   ⇒ $\boxed{4\ B\times H\_in\times S\times S}$
4) GlobalAvgPool2d (1, 1)
   - input ($B\times H\_in\times S\times S$) → ($B\times H\_in\times 1\times 1$)
   - усреднение для каждого канала
   - $B\times H\_in\times[\underbrace{S^2-1}_{\text{сложение}}+\underbrace{1}_{\text{деление}}] = B\times H\_in\times S^2$
5) ReLU, MaxPool — 0 FLOPs, так как это операции сравнения, а не арифм.

Подсчет для всей сетки:

* в pytorch профайлер считает флопсы только для матричных операций, поэтому будем учит. только linear + conv

1) Conv2d(3, 32, 7, 2, 3, B, S) $= 2352BS^2$
2) Conv2d(32, 64, 5, 1, 2, B, S/4) $= 6400BS^2$
3) Conv2d(64, 128, 3, 2, 1, B, S/4) $= 2304BS^2$
4) Conv2d(128, 256, 1, 1, 0, B, S/8) $= 1024BS^2$
5) Conv2d(256, 256, 3, 2, 1, B, S/8) $= 4608BS^2$
6) Conv2d(256, 512, 1, 1, 0, B, S/16) $= 1024BS^2$
7) Linear(512, 256) $= 2\cdot B\cdot 512\cdot 256 = 262144B$
8) Linear(256, 100) $= 2\cdot B\cdot 256\cdot 100 = 51200B$

Итог: $\boxed{B\times(17712S^2+313344)}$

Memory

Предположение: до самого forward-а мы храним вход + все веса. В реальной жизни можно так не делать.
Тогда ответом будет

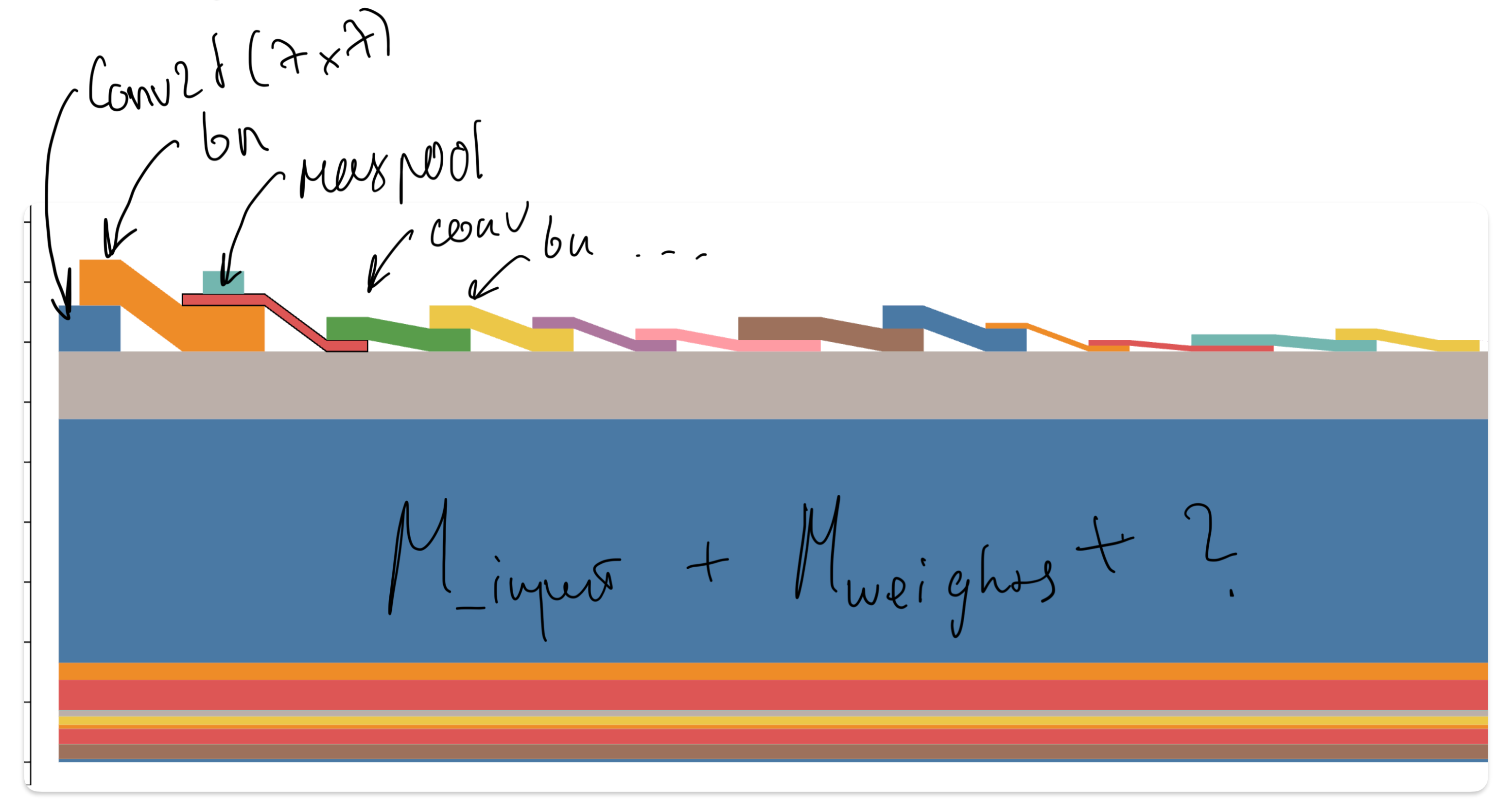
$$\boxed{M_{peak}\approx M_{input}+M_{weights}+\max_{layers}\{M\_in+M\_out\}}$$

размер входа   веса   вход и выход каждого слоя.

$M_{input} = 3\times B\times S^2\times 4$

$M_{weights} = X$ (≈ 4 млн. параметров)

Memory profiler



1) т.к. ReLU inplace ⇒ доп памяти не нужно
2) самая большая активация у bn в начале т.к. еще не было subsampling-а.

$\max_{layers}\{M_{in}+M_{out}\} = 2\times B\times 32\times\frac{S^2}{4}\times 4 = 2\times 32BS^2$

⇒ Итого: $X + 12B\times S^2 + 64B\times S^2 = X + 76B\times S^2$

Bytes moved

D — суммарно сколько прочитали и записали байт.

- Для каждого слоя будут разные значения.
  - Conv2d: $4[BCS^2+BCS^2_{out}+Ck^2]$
  - Linear: $4[B\cdot in\_f + B\cdot out\_f + in\_f(out\_f+1)]$
  - BN: $4[BCS^2+BCS^2+4C]$
  - ReLU: $4[BCS^2+BCS^2+0]$
  - MaxPool: $4[BCS^2+BCS^2_{out}+0]$
  - GlobalAvg: $4[BCS^2+BC+0]$
- Применяем к нашей сетке:

$\boxed{D(B,S) = 532BS^2 + 8592B + M_{weights}}$ ← тут в eval() ⇒ без BN

Latency

1) Есть два типа операций: 1) compute 2) communications ⇒ latency зависит от этих двух моментов.
2) compute (seconds) = FLOPs / Θ_F, где Θ_F — сколько в секунду FLOPs-ов.
3) communications (seconds) = D/Θ_D, где Θ_D — сколько в секунду можем делать операций по пересылке данных.
4) Итоговая формула.
   - Считаем, что у нас эти два типа операций выполняются параллельно, то есть достаточно смотреть на самое большое знач.

$$\boxed{latency(B,S,\Theta) = \max\left(\frac{FLOPs(B,S)}{\Theta_F},\frac{D(B,S)}{\Theta_D}\right)+b}$$

   - можно улучшить оценку введя больше число параметров:

$$latency(B,S,\Theta) = \sum_{l}\max\left(\frac{FLOPs^{(l)}(B,S)}{\Theta^{(l)}_F},\frac{D^{(l)}(B,S)}{\Theta^{(l)}_D}\right)+b$$

Energy

1) из физики: Работа = Производительность × Скорость ⇔ ⇔ Energy (Дж) = Power (ватт) × Latency (сек.)
2) так же как и с latency есть два источника на что уходят ресурсы 1) compute 2) communication.
3) Логично, что помимо этих двух факторов может быть еще много технических моментов, которые будем учитывать как E_base aka bias (latency)
4) Итоговая формула:

$$\boxed{E = FLOPs\times\Theta_F + D\times\Theta_D + latency\times\Theta_B}$$

5) Как замеряем?
   - NVML
   - Запускаем на k секунд forward-ы, потом считаем сколько потратили энергии / #fwd.